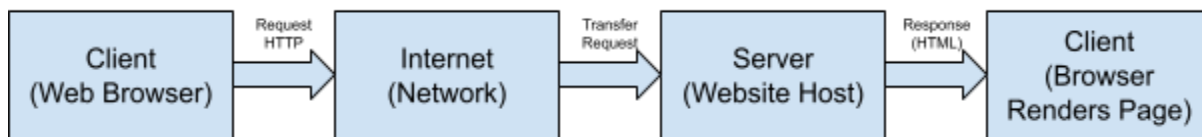


Assignment

1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

Frontend	Backend	Full-Stack
It focuses on user interface (UI), what users see and interact directly with web browsers and mobile apps. A frontend developer ensures designs should be interactive, attractive and user-friendly. Core Skills: HTML, CSS, Javascript, React, etc. Examples: A dashboard with different components like header, footer, navigation bar, progress chart, etc.	It focuses on server-side operations and databases. A backend developer handles authentication, authorization, security of data and connects frontend to database and APIs. Core Skills: Node.js, MongoDB, SQL, Python etc. Example: When we search for a product from the search bar it fetches product data from the backend and displays it to the user.	It is the combination of both frontend and backend. Build end-to-end solutions and handle UI/UX development, server and databases. Core Skills: Java, Python, SQL, Node.js, React, etc. Example: A complete website with proper UI, database and working backend.

2. Create a simple diagram showing how the client-server model works in web architecture.



3. Describe how a browser requests and displays a web page from a web server.

- User types a URL or clicks a link in the browser
- Browser sends an HTTP request to the server.
- The server processes the request and sends back the requested files.
- Browser renders the page to the user.

4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

- **Install a Code Editor**
 - Used to write, manage and edit code efficiently.
 - Example: VS code, Sublime Text, Atom, etc.
- **Install a Web Browser with Developer Tools**
 - Used to view and test how your web pages appear and behave for users.
 - Developer Tools (F12) help in inspecting elements, debugging JavaScript, and analyzing network performance.
 - Example: Google Chrome, Mozilla Firefox, etc.
- **Set Up Version Control**
 - Helps in tracking changes to your code over time.
 - Allows collaboration with other developers.
 - You can revert to previous versions or create branches for testing new features.
 - Example: Git and GitHub.
- **Package Manager**
 - Used to install and manage libraries or dependencies your project needs.
 - Saves time by avoiding manual downloads.
 - Example: [Node.js](#), NPM, etc.
- **Use a Local Development Server**
 - A server is needed to host and serve your web pages.
Local servers help you test dynamic websites (like PHP or Node.js apps) before publishing them online.
 - Example: [Node.js](#) / Express server.

5. Explain what a web server is and give examples of commonly used servers.

A web server is a software or hardware system that stores, processes, and delivers web pages to users over the internet.

When you type a URL or click a link, the web server is what receives your request and sends back the website's files (HTML, CSS, JavaScript, images, etc.) to your browser.

Examples: Apache, Nginx, IIS, LiteSpeed, Node.js, Tomcat.

6. Define the roles of a frontend developer, backend developer, and database administrator in a project.

Role	Focus Area	Main Responsibility	Key Technologies
Frontend	Client-side (User Interface)	Build what users see and interact with	HTML, CSS, JS, React

Backend	Server-side (Logic & Processing)	Handle requests, logic, and APIs	Node.js, Python, Java
DBA	Data Layer	Manage, secure, and optimize databases	MySQL, Oracle, MongoDB

8. Explain the difference between static and dynamic websites. Provide an example of each.

- **Static Website**
 - A static website displays fixed content that does not change automatically.
 - Each page is created manually using HTML, CSS, and sometimes JavaScript.
 - The same content is shown to every visitor.
 - Pages are pre-built and stored on the web server.
 - No database connection or server-side processing.
 - Faster and simpler to host.
 - Suitable for small websites or portfolio pages.
- **Dynamic Website**
 - A dynamic website displays content that changes automatically based on user actions or database data.
 - It uses server-side scripting languages and databases to generate web pages in real time.
 - Content is **fetches or updated dynamically**.
 - Uses **backend programming** (e.g., PHP, Python, Node.js).
 - Connected to a **database** (like MySQL, MongoDB).
 - Can display personalized information (e.g., user profiles, orders).
 - Suitable for **large, interactive websites**.

9. Research and list five web browsers. Explain how rendering engines differ between them.

- **Google: Blink**
 - Blink is open-source (part of Chromium). It's designed for speed and high performance. It supports modern HTML, CSS, Web APIs, etc. It uses V8 for JavaScript execution.
- **Mozilla Firefox: Gecko**
 - Gecko is Mozilla's engine, emphasizes compliance with web standards, a strong open source community. It has different trade-offs relative to Blink in layout speed, memory usage, CSS rendering, etc. It also includes specific components

like Quantum CSS and improvements to make parallelism and hardware acceleration more efficient.

- **Apple Safari: WebKit**
 - WebKit, developed by Apple from KHTML/KJS, is highly optimized for Apple hardware (macOS, iOS). Due to Apple's policies, all iOS browsers must use WebKit, which features close integration with system graphics and power considerations.
- **Microsoft Edge: Blink**
 - Edge used to use EdgeHTML (a Microsoft-proprietary engine) but since about 2019/2020 has switched to being based on Chromium, thus using Blink. This gives it compatibility with many sites optimized for Chromium, plus benefits in performance, extension ecosystem, etc.
- **Internet Explorer: Trident**
 - This is a legacy engine. Older, with many proprietary extensions, less strict compliance to modern standards than Gecko or Blink/WebKit. As web standards advanced, Trident lagged behind in modern CSS, JS features, performance. It is now mostly deprecated, but support in some cases remains for legacy sites.

Standards Support / Compliance

- How well HTML5, CSS3/4, JavaScript/ECMAScript versions, new APIs (WebGL, WebAssembly, etc.) are supported.
- Engines like Blink & WebKit are quick to adopt new standards; older ones like Trident were slower.

Performance (Speed, Rendering, Layout, Script Execution)

- How fast pages load, how smooth the rendering of animations/scrolling is.
- How efficiently JS is executed. For example, Blink uses V8 (very optimized); Gecko uses its own JS engine (SpiderMonkey), etc.

Memory & Resource Usage

- Engines may differ in how much RAM they use, how they handle GPU acceleration, how they cache content, etc.

Platform & Hardware Integration

- WebKit is tightly integrated with Apple hardware/software (macOS, iOS).
- Blink/Chromium is designed to be cross-platform (Windows, Linux, Android, etc.).
- Some engines have constraints (e.g. on iOS, non-Safari browsers must use WebKit underneath).

Legacy / Backward Compatibility

- Engines like Trident/EdgeHTML maintained backward compatibility for old web pages/sites.
- Newer engines tend to drop older deprecated features but aim for consistency and modern features.

Security and Updates

- How frequently bugs (including rendering bugs, script vulnerabilities) are patched.
- How the engine architecture isolates or sandbox components (rendering vs JS vs UI vs network) to reduce risk.

10. Draw a labeled diagram showing the basic web architecture flow — client, server, database, and APIs.

